

CSS fingerprinting is a browser fingerprinting technique that allows a website to identify and track visitors using CSS. CSS fingerprinting is stateless, that is, it does not require the computer system running the browser to remember information. It leverages differences across browsers, or CSS queries that allow programmers to target different systems, to infer information about a user's system. Since CSS is typically allowed in areas where JavaScript code execution is disabled, such as in email clients, it has a larger reach than most browser fingerprinting techniques, which typically rely on JavaScript code.

Background

Browser fingerprinting is a set of techniques that leverage a browser's unique characteristics and features to track a user. Unlike stateful tracking through the use of cookies, browser fingerprinting collects small inconsistencies and quirks in a particular browser's behavior, which are often a function of the hardware the browser is being run on and configurations that are being used on the browser. These inconsistencies are used to build a unique identifier for a particular user. Typically this process requires the use of JavaScript code which subsequently calls a variety of HTML5 APIs to generate a unique identifier. As a result of this, modern fingerprinting defenses limit the amount and types of JavaScript code that a website can run. In certain cases, extensions like NoScript outright disallow the execution of JavaScript code.

CSS fingerprinting allows a website to fingerprint a user without executing any JavaScript code, thus bypassing traditional fingerprinting defenses. Since style sheets are typically allowed in user-generated content, even where script-based JavaScript execution was disallowed (such as email clients), CSS fingerprinting has a larger reach than traditional browser fingerprinting.

Techniques

CSS fingerprinting techniques are typically dependent on abusing specific features of CSS to detect attributes about the system that CSS would not ordinarily be able to access. The process usually consists of two parts: the initial access to the information on the user's system, and the subsequent exfiltration of that information. While there are many ways of gaining access to data through CSS fingerprinting, exfiltrating it purely through CSS is difficult since the browser does not allow the dynamic generation of URLs in CSS. As a result, CSS fingerprinting typically relies on making a conditional networking request. Examples include using a background image on a CSS selector, a font retrieval on a media query, and using a conditional image-set directive.

One particular method of CSS-based fingerprinting is font fingerprinting, which focuses on detecting the presence of specific fonts on a user's system. By revealing the presence of certain fonts, a website can infer whether certain applications have been installed by the user. This involves the website loading a font using the `@font-face` CSS directive, or directly loading a CSS font by applying the `font-family` directive to an HTML element. By measuring the size of the element when the `font-family` directive is applied directly, a website can infer the presence of system fonts and other applications. Another font-based attack proposed by Heiderich et al. in 2012 centered around loading specially prepared fonts where the glyphs for a set of letters had been swapped out for zero-width glyphs. Once these fonts have loaded, an attacker can apply these fonts on an element containing data they want to leak. Using a series of CSS animations and height/width queries, the attacker can infer the heights of the characters which are not zero-width characters making it possible to leak the data contained in the element character by character.

Media queries are a set of CSS directives that allow a website to query different properties about a screen such as the height, width or whether the user is in a printing interface. Media queries allow websites to conditionally apply CSS styles if a particular set of conditions are met. Other techniques include using CSS's in-built `calc()` functionality to perform calculations

that reveal the instruction set and precision of the underlying operating system, and using CSS's `@supports` queries to check if particular CSS features are present in a browser, which can be used to identify the specific browser and version. Other features available to CSS include whether or not JavaScript is enabled and scrollbar settings, particularly on macOS.

Another technique that can be used to fingerprint users is extension detection. Extension detection is a technique that uses CSS queries to determine the presence or absence of specific attributes and elements injected by browser extensions. Certain extensions, such as Wikiwand, modify web pages by injecting CSS style sheets and additional elements to alter their behavior. An attacker can exploit this by setting up a website that employs CSS container queries to detect these modifications. By analyzing the results of these queries and comparing them against a previously compiled database of extension fingerprints, an attacker can identify which extensions a user has installed. This information can allow the attacker to uniquely identify users based on their installed extensions.

CSS fingerprinting techniques can also be used to dynamically exfiltrate user-interactions. An attacker can craft code to register a CSS selector that gets activated only when a user types a specific string into an input field, areas that might reveal contextual information about the page where the CSS was loaded. The CSS selector subsequently embeds a background-image on an element that is invisible to the user, with the URL request exfiltrating the data on the user interaction to the attacker. Depending on the amount of CSS being loaded, the malicious CSS could also be used as a keylogger that returns a user's keystrokes into specific sensitive fields.

Example

The following example demonstrates how CSS fingerprinting could be used to infer information about a user's browser configuration.

In the example, a set of CSS directives are declared that use media queries. The CSS directives apply different types of backgrounds on to the body element depending on the result of queries on the width of a user's screen. The background in turn makes a request to a remote URL for that background image, with the choice of image conveying information about the type of device. The remote URL can now collect information on which device was used based on which image was requested.

References

Citations

References

Further reading

Hayes, Gareth (2025-08-26). "Inline Style Exfiltration: leaking data with chained CSS conditionals". PortSwigger Research. Archived from the original on 2026-01-16. Retrieved 2026-01-18.

Gualtieri, Mike (2018-02-06). "Stealing Data With CSS: Attack and Defense | Mike Gualtieri". Mike Gualtieri's blog. Retrieved 2026-01-18.